CASE 05 / PUBLIC SDK | NATIVE ENGINES & BRIDGES

Offline Location Sync Native SDK & Bridges

Native Swift/Kotlin engines with Flutter and React Native integration layers.

Swift | Kotlin | Flutter | React Native | API Integration

PROBLEM

Location features become difficult when an app collects records without a stable connection and later synchronizes them. Teams also need integration points that respect native platform behavior. I built RTLS as a public SDK project exploring these concerns through native engines, persistence, synchronization policies, and cross-platform integration layers.

CONTRIBUTION

The project separates location collection, offline synchronization, shared contracts, and WebSocket messaging, with an optional coordinating client. Native Swift and Kotlin engines provide the platform foundations. Supporting work includes SQLite persistence, configurable batch uploads and retry policies, a Node.js backend contract, and a React map dashboard for inspecting the data flow.

DECISIONS

The iOS implementation uses Core Location, an actor-based synchronization engine, URLSession, and background task integration. Android modules separate common policies from platform implementations for storage, transport, connectivity, and location. Flutter includes legacy native bridges and newer modular Dart facades; React Native provides TypeScript APIs and native event delivery. These surfaces wrap native capabilities rather than representing four independent production engines.

RESULT

The result is an inspectable SDK and integration reference for discussing telemetry, offline data, and native bridging. Modular boundaries allow teams to reason separately about collecting records, retaining pending data, and communicating with a backend. Documentation covers linking, permissions, configuration, and migration. Client integration includes selecting suitable packages and validating platform setup and background execution behavior; the portfolio scope is the public implementation and reference architecture.

ENGINEERING FLOW

Collect & Persist
Native location modules and SQLite storage

→

Batch & Retry
Configurable synchronization and transport policies

→

Framework APIs
Dart and TypeScript integration surfaces

EVIDENCE

- RTLS - public SDK project and platform overview
- Modular architecture documentation
- Flutter integration documentation
- React Native integration reference